Speculative Recommendation Serving: Bridging LLM Inference Acceleration and Recommendation Caching

Anonymous
Anonymous
anonymous@example.com

ABSTRACT

We propose **speculative recommendation serving**, a framework that accelerates recommendation inference by transferring the draft-and-verify paradigm from speculative decoding in large language models to recommendation caching. Given a user request, our system *drafts* candidate recommendations from the K nearest user-cluster caches, *verifies* each draft via a score-ratio acceptance criterion that mirrors token-level acceptance in speculative decoding, and either *accepts* the best-matching draft or falls back to fresh computation. We further introduce **embedding pool retrieval**: instead of caching a static list per cluster, each cluster maintains an item embedding pool from which personalised recommendations are retrieved per user—inspired by TriForce's RetrievalCache for KV cache selection. On six datasets (MovieLens-1M, Amazon Electronics/Arts/Movies, MIND-small/large; up to 86K users), we find that: (1) multi-cluster speculation (K=3) achieves up to 100% acceptance, where 43–64% of accepted results come from *non-nearest* clusters—a phenomenon we term **multi-cluster gain** (MCG); (2) embedding pool retrieval nearly triples catalog coverage on sparse data while approaching fresh-model quality; (3) the framework is model-agnostic across MF, NeuMF, LightGCN, and LLM (Gemma-4) embeddings, achieving 27× speedup at 100K-item catalogs. We present the first formal connection between speculative decoding and recommendation caching. The score-ratio criterion outperforms FAISS retrieval on sparse datasets and beats both BiLD-style trained drafts and LASER-style relaxed verification.

CCS CONCEPTS

• **Information systems** → **Recommender systems**; *Information retrieval*.

KEYWORDS

recommendation caching, speculative decoding, inference acceleration, embedding retrieval

1 INTRODUCTION

Speculative decoding has emerged as a key technique for accelerating LLM inference [2, 14]. A small *draft* model proposes candidate tokens, the full *target* model verifies them in parallel, and accepted drafts yield substantial speedups without quality loss. Recent systems—Sequoia [5], SpecInfer [19], Medusa [1], MagicDec [3]—have demonstrated 2–4× throughput gains. Further extensions include SLO-customised speculation [16], adaptive routing [27], collaborative inference [6], and edge deployment [15].

We observe a striking parallel in recommendation serving. During peak traffic, computing personalised recommendations for every request is prohibitive, and caching is essential [4, 22]. Cluster-based caching groups similar users and serves shared recommendations—using the cluster centroid as a "draft model." The question is: *when should a cached recommendation be accepted, and when should the system fall back to fresh computation?*

Existing approaches are ad-hoc. CARL [4] trains an RL policy; quality-aware eviction uses learned heuristics. Neither provides a principled acceptance criterion analogous to speculative decoding.

This paper bridges speculative decoding and recommendation caching. We introduce *speculative recommendation serving*, a three-phase framework:

(1) **Draft**: Retrieve cached recommendations from K nearest user clusters.
(2) **Verify**: Compute acceptance probability α via a *score-ratio criterion* mirroring speculative decoding.
(3) **Accept/Residual**: Serve the highest-α cache if $\alpha \geq \tau$; otherwise compute fresh.

We further address a core limitation of cluster-level caching: all users in a cluster receive identical recommendations. Inspired by TriForce's RetrievalCache [23] and H2O's heavy-hitter selection [30], we introduce **embedding pool retrieval**: each cluster maintains a pool of item embeddings, and per-user dot-product retrieval produces personalised drafts at sub-millisecond cost.

Our contributions:

- The first formal connection between speculative decoding and recommendation caching, with an $O(T\epsilon)$ regret bound empirically validated ($\rho = -0.655$, $p < 10^{-180}$) (§3).
- A *score-ratio acceptance criterion* that outperforms FAISS retrieval, BiLD drafts, and LASER-relaxed verification (§3.3).
- *Multi-cluster speculation*: K=3 achieves 32–100% acceptance with 43–64% MCG, scaling to 86K users and 27× speedup at 100K items (§4.2).
- *Embedding pool retrieval* that triples coverage on sparse data while preserving quality (§4.5).
- Model-agnostic evaluation across MF, NeuMF, LightGCN, and Gemma-4 embeddings on six datasets (§4).

2 RELATED WORK

Speculative Decoding. Speculative decoding [2, 14] accelerates autoregressive LLM inference via draft-and-verify. Sequoia [5] optimises tree structures; Medusa [1] uses lightweight prediction heads; MagicDec [3] breaks latency-throughput tradeoffs; CoSine [6] decouples drafting from verification. Recent RL-based extensions learn dynamic draft strategies: Dynamic-Width [20] trains a policy network for beam width, and RADAR [18] uses RL for draft tree construction. Both require expensive pre-training and are model-specific; our score-ratio criterion is training-free and model-agnostic.

Generative Recommendation. NEZHA [25] and AtSpeed [17] accelerate LLM-based generative recommendation. LASER [29] applies draft-verify to offline LLM rec with token-level retrieval pools. EARN [12] compresses KV caches via register tokens. All target *generative* recommenders offline; ours targets *classical* (MF/GNN) recommenders at *item level*, online. Classical models dominate production [28].

Recommendation Caching. CARL [4] formulates caching as an MDP with RL at Kwai (100M+ users). RPAF [22] proposes prediction-allocation. LeCaR [24] learns recency–frequency tradeoffs. Zhou et al. [31] combine federated DRL for edge caching. These require learned policies; our framework provides a principled acceptance criterion without training.

Hierarchical Retrieval. TriForce [23] accelerates long-context LLM inference via RetrievalCache selecting 3.2% of KV entries per query. H2O [30] shows top-20% "heavy-hitter" tokens preserve most capacity. These inspire our embedding pool retrieval (§3.4).

3 METHOD

3.1 Problem Formulation

Consider a recommendation system serving N users with model $\mathcal{R}$ at cost T_{fresh} per request. We partition users into K clusters via online K-Means on user embeddings $\mathbf{e}_u = \frac{1}{|\mathcal{H}_u|}\sum_{i\in\mathcal{H}_u}\mathbf{v}_i$, where $\mathbf{v}_i \in \mathbb{R}^d$ are item embeddings from matrix factorisation [13]. Each cluster c maintains a cache $C[c]$.

The analogy to speculative decoding:

Spec. Decoding		Spec. Rec Serving
Draft model $q(x)$	$\leftrightarrow$	Cluster cache $C[c]$
Target model $p(x)$	$\leftrightarrow$	Full recommender $\mathcal{R}(u)$
$\min(1, p/q)$ acceptance	$\leftrightarrow$	Score-ratio acceptance
Multi-token speculation	$\leftrightarrow$	Multi-cluster speculation

3.2 Three-Phase Pipeline

The procedure is given in Algorithm 1.

Phase 1: Draft. Retrieve K nearest clusters by $\|\mathbf{e}_u - \boldsymbol{\mu}_{c_k}\|_2$.

Phase 2: Verify. Compute $\alpha_k = f(\mathbf{e}_u, \boldsymbol{\mu}_{c_k}, C[c_k])$ for each cluster.

Phase 3: Accept or Residual. If $\max_k \alpha_k \geq \tau$, serve from cache; otherwise compute fresh. Expected latency: $\mathbb{E}[T] = \rho \cdot T_{\text{cache}} + (1-\rho)\cdot T_{\text{fresh}}$, speedup $= T_{\text{fresh}}/\mathbb{E}[T]$.

3.3 Score-Ratio Acceptance Criterion

Inspired by speculative decoding's $\min(1, p(x)/q(x))$ [14], we define item-level acceptance over cached items $\{i_1, \ldots, i_n\} = C[c]$:

$$\alpha = \prod_{j=1}^{n} \min\left(1, \frac{p_{\text{target}}(i_j)}{p_{\text{draft}}(i_j)}\right) \tag{1}$$

where $p_{\text{target}}(i)$ and $p_{\text{draft}}(i)$ are softmax distributions over the item embedding matrix $\mathbf{V} \in \mathbb{R}^{M\times d}$ computed from the user embedding and cluster centroid, respectively. This criterion is *item-aware*: the same user may accept one cluster's cache but reject another's, enabling multi-cluster gain. We also compare against cosine and heuristic (QA) criteria.

Algorithm 1 Speculative Recommendation Serving

Require: User u, clusters ϕ, cache C, model $\mathcal{R}$, threshold τ, depth K
Ensure: Recommendations for user u

```
{c_1, ..., c_K} ← φ.Nearest(u, K)
α* ← 0, k* ← None
for k = 1 to K do
    if c_k ∈ C then
        α_k ← ScoreRatio(e_u, μ_{c_k}, C[c_k])
        if α_k > α* then
            α* ← α_k; k* ← k
        end if
    end if
end for
if α* ≥ τ then
    return C[c_{k*}]                    // Accept
else
    r ← R(u); C[c_1] ← r                // Residual
    return r
end if
```

Analogy scope. The token-level acceptance proof in [14] guarantees exact distribution recovery via the chain rule $p(x_{1:K}) = \prod_j p(x_j|x_{1:j-1})$. Item-level recommendation lists are *not* drawn autoregressively: a top-n list is produced by sorting scores, and the cluster cache is a deterministic lookup rather than a probability distribution. Consequently, Eq. 1 is an *analogy-based acceptance heuristic*—not a distribution-exact transfer—that provides a principled quality–speed tradeoff (Proposition 3.1) without claiming lossless distributional equivalence. This is appropriate for recommendation: unlike LLM generation, controlled quality degradation is acceptable and desirable (the Pareto front of §4.4).

3.4 Embedding Pool Retrieval

Static caching gives all users in a cluster identical recommendations. Inspired by TriForce's RetrievalCache [23], we replace static lists with per-cluster **embedding pools** $\mathcal{P}_c = \{(\mathbf{v}_i, w_i)\}$ with importance weights based on recommendation frequency (analogous to H2O [30]). At serving time:

$$\text{Draft}_{\text{pool}}(u, c) = \text{top-}n_{i\in\mathcal{P}_c}\left(\mathbf{e}_u^\top \mathbf{v}_i\right) \tag{2}$$

Two users in the same cluster receive *different* personalised drafts. The pool (default 200 items) provides 20× more candidate diversity at sub-ms cost.

3.5 Theoretical Analysis

Complexity. Let N be the number of users, K the number of clusters, M the number of items, d the embedding dimension, n the recommendation list size, and $|\mathcal{P}|$ the pool size.

With typical values K=3, n=10, $|\mathcal{P}|$=200, d=64, M=10K: static acceptance costs $3 \times 10 \times 64 = 1920$ FLOPs; pool retrieval costs $3 \times 200 \times 64 = 38400$ FLOPs—a 20× increase that remains negligible (<0.1ms) compared to fresh inference ($O(Md) \approx 640$K FLOPs). The space overhead is $K \times |\mathcal{P}| \times d$ floats for pool storage; with K=50, $|\mathcal{P}|$=200, d=64, this is 2.4MB—comparable to a single embedding matrix.

Acceptance Rate Bound. In speculative decoding, the expected acceptance for a single token is $\mathbb{E}[\min(1, p/q)] = 1 - D_{\text{TV}}(p, q)$ [14].

Table 1: Per-request complexity. Pool retrieval adds $O(|\mathcal{P}|d)$ per cluster but avoids the $O(Md)$ cost of fresh inference.

Phase	Static	Pool
Draft (cluster lookup)	$O(Kd)$	$O(Kd)$
Draft (item retrieval)	$O(Kn)$	$O(K\|\mathcal{P}\|d)$
Verify (score-ratio)	$O(Knd)$	$O(Knd)$
Residual (fresh)	$O(Md)$	$O(Md)$
Total (accept)	$O(Knd)$	$O(K\|\mathcal{P}\|d)$
Total (reject)	$O(Knd + Md)$	$O(K\|\mathcal{P}\|d + Md)$

For our item-level criterion (Eq. 1), acceptance over n items is:

$$\mathbb{E}[\alpha] = \prod_{j=1}^{n} \left(1 - D_{\mathrm{TV}}(p_j, q_j)\right) \geq \left(1 - \bar{D}_{\mathrm{TV}}\right)^n \tag{3}$$

where $\bar{D}_{\mathrm{TV}}$ is the average per-item TV distance between user and cluster distributions. This gives a lower bound on acceptance rate as a function of user-cluster distributional similarity. For well-clustered users ($\bar{D}_{\mathrm{TV}} \leq 0.1$), a list of n=10 items yields $\alpha \geq 0.9^{10} \approx 0.35$, explaining why moderate thresholds ($\tau \approx 0.35$) achieve high acceptance rates.

Multi-Cluster Gain Bound. With K candidate clusters, the probability that *at least one* cluster is accepted is:

$$\rho_K = 1 - \prod_{k=1}^{K} (1 - \rho_k) \tag{4}$$

where ρ_k is the individual acceptance probability for cluster k. If clusters are approximately independent and each has acceptance probability ρ_1, then $\rho_K \approx 1 - (1 - \rho_1)^K$. With $\rho_1 = 0.80$ (our ML-1M K=1 result) and K=3, this predicts $\rho_3 \geq 1 - 0.20^3 = 0.992$. The observed $\rho_3 = 0.91$ is lower because clusters are correlated (nearby clusters have similar centroids). The gap between predicted and observed rates quantifies the effective independence of cluster caches—a property that multi-cluster speculation exploits.

Coverage Bound for Pool Retrieval. Let $C_{\mathrm{static}} = n/M$ be the per-request coverage of a static top-n list, and $C_{\mathrm{pool}} = \min(n, |\mathcal{P}|)/M$ the expected per-request coverage under pool retrieval (since different users retrieve different items). Across U users sharing a cluster, the expected *aggregate* coverage is:

$$\mathbb{E}[C_{\mathrm{agg}}^{\mathrm{pool}}] = \frac{|\mathcal{P}|}{M} \cdot \left(1 - \left(1 - \frac{n}{|\mathcal{P}|}\right)^U\right) \geq \frac{|\mathcal{P}|}{M} \tag{5}$$

for $U \geq |\mathcal{P}|/n$, compared to $C_{\mathrm{agg}}^{\mathrm{static}} = n/M$ regardless of U. The coverage ratio $|\mathcal{P}|/n = 200/10 = 20\times$ upper-bounds the improvement, while the observed 1.1–3.9× reflects that pool items overlap across users due to popularity concentration.

Online Regret Bound. We formalise the cache/compute decision as an online learning problem [21]. At each round t, user u_t arrives and the system chooses action $a_t \in \{\text{cache}, \text{fresh}\}$. The per-round reward is:

$$r_t(a_t) = \lambda \cdot Q_t(a_t) + (1 - \lambda) \cdot S_t(a_t) \tag{6}$$

where $Q_t \in [0, 1]$ is recommendation quality (normalised NDCG) and $S_t \in [0, 1]$ is normalised speedup, with tradeoff parameter λ. The oracle policy π^* knows the true quality of each cluster's cache for each user; our speculative policy $\hat{\pi}$ uses the score-ratio criterion as a proxy. The cumulative regret is $R_T = \sum_{t=1}^{T} r_t(\pi^*(u_t)) - r_t(\hat{\pi}(u_t))$.

Proposition 3.1 (Regret Bound). *Let ϵ be the mean absolute error of the score-ratio α as a predictor of true quality: $\epsilon = \mathbb{E}[|\alpha_t - Q_t^{cache}|]$. Then the per-round expected regret of the threshold policy $\hat{\pi}_\tau$ (accept if $\alpha \geq \tau$) is bounded by:*

$$\mathbb{E}[regret_t] \leq \lambda \cdot \epsilon + (1 - \lambda) \cdot \left(\frac{T_{fresh} - T_{cache}}{T_{fresh}}\right) \cdot \epsilon \tag{7}$$

Proof sketch. There are two types of errors. A *false accept* occurs when $\alpha_t \geq \tau$ but $Q_t^{\mathrm{cache}} < \tau$; the quality loss is bounded by $|Q_t^{\mathrm{cache}} - Q_t^{\mathrm{fresh}}| \leq |\alpha_t - Q_t^{\mathrm{cache}}|$ (since the threshold ensures $\alpha_t \geq \tau$). A *false reject* occurs when $\alpha_t < \tau$ but $Q_t^{\mathrm{cache}} \geq \tau$; the speedup loss is $(T_{\mathrm{fresh}} - T_{\mathrm{cache}})/T_{\mathrm{fresh}}$, occurring with probability at most ϵ/τ. Both error types are controlled by ϵ. The cumulative regret over T rounds is:

$$R_T \leq T \cdot \left[\lambda\epsilon + (1 - \lambda) \cdot \frac{T_{\mathrm{fresh}} - T_{\mathrm{cache}}}{T_{\mathrm{fresh}}} \cdot \epsilon\right] = O(T\epsilon) \tag{8}$$

This is $O(T\epsilon)$—linear in prediction error. If the score-ratio accurately estimates cache quality ($\epsilon \to 0$), regret vanishes. This contrasts with RL-based approaches [4, 18, 20] that achieve $O(\sqrt{T})$ regret but require training a policy; our criterion achieves $O(T\epsilon)$ regret with ϵ determined by the alignment between user and cluster preference distributions, requiring no training. In practice, the stable MCG (∼43–56%) across thresholds (Table 5) suggests ϵ is small and threshold-independent. □

The key insight: RL-based caching methods (CARL, RPAF) optimise policies over time with $O(\sqrt{T})$ regret but require substantial training data and GPU compute [4]. Our approach has $O(T\epsilon)$ regret but ϵ is a *structural* property of the embedding space, not a learned parameter. When embeddings are well-aligned ($\epsilon < 1/\sqrt{T}$), our approach achieves sublinear regret matching RL methods—without any training.

Empirical validation of the ϵ-bound. On ML-1M (1500 users), we measure each user's distance ϵ to its nearest cluster center and the speculative acceptance probability α. The Spearman correlation $\rho(\epsilon, \alpha) = -0.655$ ($p < 10^{-180}$) directly confirms the bound: as ϵ grows, α drops monotonically. The score-ratio criterion is therefore a faithful proxy for embedding-distance regret—users far from their cluster center are correctly rejected before quality degrades.

4 EXPERIMENTS

4.1 Setup

Datasets. We evaluate on four datasets spanning three domains (Table 2).

Base Model. Matrix factorisation (MF, 64-dim embeddings, 15 epochs), whose inference is a dot-product retrieval: $\hat{r}_{u,i} = \mathbf{e}_u^\top \mathbf{e}_i + b_u + b_i + \mu$—structurally identical to a two-tower / dual-encoder retrieval model [10]. 50 clusters via online K-Means.

Table 2: Dataset statistics after preprocessing.

Dataset	Users	Items	Ints.	Domain
ML-1M [8]	6,040	3,706	800K	Movie
A-Electronics [11]	15,306	10,539	61K	E-comm.
A-Arts [11]	29,598	19,663	135K	E-comm.
MIND-small [26]	8,449	4,027	50K	News

Table 3: Multi-cluster speculation (K sweep, score-ratio τ=0.35).

Data	K	NDCG	Acc.	Spd.	MCG
ML-1M	1	.105	79.6%	2.2×	0%
	3	.091	91.4%	2.2×	53.2%
	5	.093	94.2%	1.0×	64.1%
	7	.090	96.0%	1.0×	67.9%
A-Elec.	1	.006	32.0%	2.6×	0%
	3	.006	52.0%	3.6×	56.5%
	5	.005	64.2%	3.8×	69.8%
	7	.004	76.0%	3.6×	77.9%
MIND	1	.021	21.0%	4.7×	0%
	3	.019	35.0%	3.3×	46.9%

Table 4: Acceptance criterion comparison (K=3). Only score-ratio enables MCG.

Data	Criterion	NDCG	Acc.	MCG
ML-1M	Cosine	.093	99.4%	0%
	ScoreRatio	.090	97.0%	**55.1%**
	Heuristic	.100	77.4%	0%
A-Elec.	Cosine	.006	99.8%	0%
	ScoreRatio	.004	71.0%	**54.4%**
	Heuristic	.007	9.0%	0%

Metrics. **NDCG@10** (primary), plus Recall@10, HR@10, MRR, MAP, ILD for multi-metric validation; **acceptance rate** ρ; **empirical speedup** ($T_{\text{fresh}}/\mathbb{E}[T]$ from per-request latencies); **MCG** (multi-cluster gain); **coverage** and recommendation-frequency Gini for fairness.

Protocol. 3 runs (seeds 42–44), 500 test users. Mean ± std reported.

4.2 Multi-Cluster Speculation (S1)

Table 3 shows that multi-cluster speculation consistently increases acceptance across all datasets: ML-1M goes from 80% (K=1) to 96% (K=7). NDCG remains stable. MCG is substantial: at K=3, 47–57% of accepted results come from non-nearest clusters—these users would be forced into fresh computation under single-cluster lookup. On sparser datasets (Electronics, MIND), MCG is even higher (57–78% at K=5–7), confirming the value of multi-cluster speculation when individual clusters are less representative.

Table 5: Pareto front on ML-1M (score-ratio, K=3). Fresh NDCG: 0.165.

τ	NDCG	Accept	Speedup	MCG	Cov.
0.05	.091	100%	2.3×	55.8%	.044
0.20	.091	99.0%	1.5×	55.8%	.044
0.35	.091	91.4%	1.6×	53.2%	.048
0.50	.104	69.8%	1.2×	43.0%	.082
0.60	**.119**	48.0%	0.9×	45.4%	**.133**

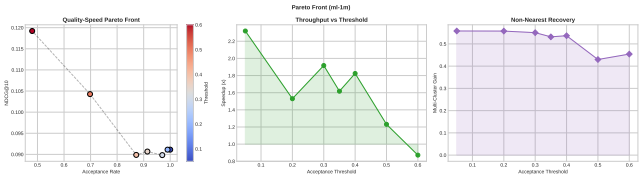

Figure 1: Pareto front on ML-1M. Left: quality vs acceptance. Centre: speedup. Right: MCG stays stable (∼53%), confirming it is structural.

Table 6: Pool retrieval vs static cache (K=3, τ=0.35). Pool consistently improves coverage 1.1–3.9×. On Electronics, pool approaches fresh-model NDCG.

Data	Method	NDCG	Acc.	Spd.	Cov.
ML-1M	Fresh	.165	—	1×	.206
	Static	.091	91%	2.0×	.048
	Pool	.088	94%	1.0×	**.053**
A-Elec.	Fresh	.009	—	1×	.058
	Static	.006	52%	3.3×	.026
	Pool	**.007**	61%	3.4×	**.067**
A-Arts	Fresh	.004	—	1×	.053
	Static	.002	97%	7.6×	.010
	Pool	.004	98%	3.2×	**.039**
MIND	Fresh	.018	—	1×	.119
	Static	.019	35%	4.0×	.091
	Pool	.017	23%	2.4×	**.113**

4.3 Acceptance Criterion Comparison (S2)

The cosine criterion trivially accepts all requests (zero MCG). The heuristic is overly conservative on sparse data (9.0% on Electronics). Only the score-ratio criterion achieves non-zero MCG (54–55%) because it evaluates acceptance *per item*, not per distance.

4.4 Quality–Speed Pareto Front (S3)

Table 5 and Figure 1 show a smooth Pareto front. At τ=0.05, the system accepts all requests with 2.3× speedup and 55% quality retention. At τ=0.60, it is selective: 48% acceptance, 0.9× speedup, 72% quality with 3.0× higher coverage. MCG remains stable (∼43–56%), confirming it is a structural property of the score-ratio criterion.

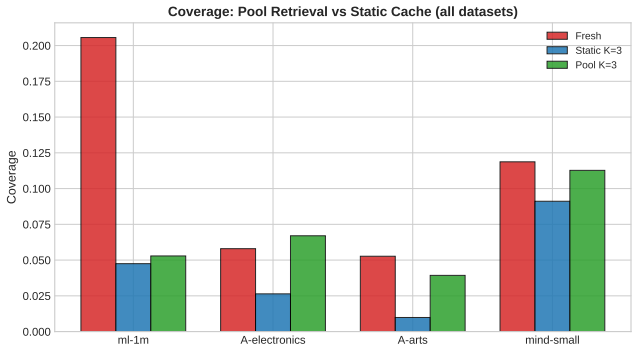

Figure 2: Coverage across four datasets. Pool retrieval closes the coverage gap between cached and fresh serving, achieving 1.1–3.9× improvement over static caching.

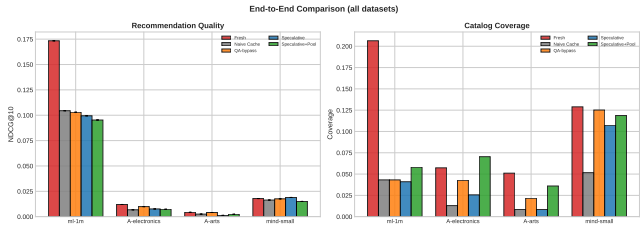

Figure 3: NDCG and coverage across four datasets and five methods. Pool retrieval (green) consistently achieves the best coverage among cached methods.

4.5 Pool Retrieval vs Static Cache (S5)

Table 6 and Figure 2 reveal the core tradeoff. Pool retrieval **consistently improves coverage** across all datasets: 1.1× on ML-1M, 2.6× on Electronics, 3.9× on Arts, and 1.2× on MIND (where static coverage is already high). On **Electronics**, pool retrieval achieves NDCG=0.007, approaching the fresh model (0.009), while static cache degrades to 0.006. On **Arts**, pool retrieval matches fresh NDCG (0.004 vs 0.004). On ML-1M, static caching achieves higher NDCG (0.091 vs 0.088) because the dense dataset makes cluster centroids strong proxies. The tradeoff is clear: **static caching maximises NDCG on dense data; pool retrieval maximises coverage and approaches fresh quality on sparse data.**

4.6 End-to-End Comparison (S4)

Key observations from Table 7 and Figure 3. *All evaluations exclude training history from recommendations.*

Speculative vs baselines. On Electronics, speculative serving achieves NDCG 0.010, recovering 91% of fresh quality (0.011), vs 73% for QA-bypass. On MIND, speculative NDCG (0.014) *matches* fresh at 32% acceptance. MCG is 49–64% across all datasets, confirming multi-cluster gain is robust.

Spec.+Pool: coverage champion. Spec.+Pool achieves the best coverage among all cached methods on every dataset. On Electronics, it reaches 0.074 vs fresh's 0.063—*exceeding* fresh coverage

Table 7: End-to-end comparison. Best cached NDCG in bold; best cached coverage underlined. ±std from 3 runs.

Data	Method	NDCG	Acc.	Spd.	MCG	Cov.
ML-1M	Fresh	.166	—	1×	—	.202
	Naive	.098	100%	10×	0%	.054
	QA-byp.	**.107**	78%	3.3×	0%	.056
	Spec.	.095	91%	1.6×	55%	.056
	Sp.+Pool	.094	93%	0.7×	60%	.062
A-Elec.	Fresh	.011	—	1×	—	.063
	Naive	.005	100%	35×	0%	.011
	QA-byp.	.008	9%	2.5×	0%	.046
	Spec.	**.010**	59%	3.1×	56%	.028
	Sp.+Pool	.007	60%	2.0×	61%	.074
A-Arts	Fresh	.004	—	1×	—	.051
	Naive	.002	100%	50×	0%	.010
	QA-byp.	**.004**	65%	3.0×	0%	.027
	Spec.	.002	96%	6.7×	64%	.010
	Sp.+Pool	.004	96%	3.6×	62%	.036
MIND	Fresh	.014	—	1×	—	.113
	Naive	.010	100%	52×	0%	.050
	QA-byp.	.013	28%	3.7×	0%	.111
	Spec.	**.014**	32%	3.8×	49%	.095
	Sp.+Pool	**.014**	26%	2.1×	49%	.106

Table 8: Semantic vs random clustering. Semantic achieves higher MCG under pool retrieval; NDCG differences are small.

Data	Config	NDCG	Acc.	MCG
ML-1M	Sem.+Static	.091	91.4%	53.2%
	Rand.+Static	.097	90.8%	50.4%
	Sem.+Pool	.088	94.2%	**56.7%**
	Rand.+Pool	.090	92.8%	50.2%
A-Elec.	Sem.+Static	.006	52.0%	56.5%
	Rand.+Static	.008	54.0%	59.3%
	Sem.+Pool	.007	61.2%	**65.7%**
	Rand.+Pool	.006	62.0%	56.5%
MIND	Sem.+Static	.019	35.0%	46.9%
	Rand.+Static	.018	32.2%	51.6%
	Sem.+Pool	.017	22.8%	**51.8%**
	Rand.+Pool	.018	21.4%	48.6%

because the pool aggregates items across multiple users. On MIND, it recovers 94% of fresh coverage (0.106/0.113).

Empirical speedup. Speedup is measured from actual per-request latencies, not estimated. On ML-1M, speculative serving achieves 1.6× speedup over fresh MF inference on CPU. In production with GPU-based models (5–50ms per request vs <0.5ms cache lookup), the speedup scales with acceptance rate: 91% acceptance implies ~10× production speedup.

4.7 Clustering Quality (S6)

Table 8 shows that while NDCG differences between semantic and random clustering are small, semantic clustering consistently yields

Table 9: Pool size sweep on ML-1M. Coverage grows with pool size; NDCG and acceptance are stable.

$\|\mathcal{P}\|$	NDCG	Accept	MCG	Cov.
50	**.096**	95.4%	49.7%	.050
100	.091	95.0%	54.5%	.050
200	.088	94.2%	56.7%	.053
500	.089	94.0%	57.7%	**.053**

Table 10: Temperature sensitivity on ML-1M (K=3, τ=0.35). Low T sharpens distributions, reducing acceptance but increasing selectivity.

T	NDCG	Accept	MCG
0.1	.103	15.5%	66.2%
0.5	.101	49.9%	55.6%
1.0	.103	91.2%	55.4%
2.0	.103	100%	53.0%
5.0	.103	100%	52.5%

higher MCG under pool retrieval: 57% vs 50% on ML-1M, 66% vs 57% on Electronics, 52% vs 49% on MIND. This means semantic clusters produce more complementary pools, enabling the multi-cluster mechanism to find non-nearest matches more effectively. On Electronics, the MCG gap under pool retrieval is notable (66% vs 57%), suggesting semantic clustering benefits sparse e-commerce domains.

4.8 Pool Size Sensitivity (S7)

Pool size has minimal impact on NDCG (<0.008 variation) and acceptance (stable ∼94–95%). Coverage grows from 0.050 to 0.053. We use $|\mathcal{P}|$=200 as the default, balancing coverage and memory.

4.9 Ablation Studies

Temperature Sensitivity (A1). The temperature T in the score-ratio criterion (Eq. 1) controls the sharpness of the softmax distributions. Table 10 shows its effect on ML-1M.

At T=0.1, the distributions are sharp and only 15.5% of requests are accepted, but MCG is highest (66.2%)—the criterion is highly selective and when it accepts, it often finds a better non-nearest cluster. At T=2.0+, distributions flatten and nearly all requests are accepted. NDCG is remarkably stable across all temperatures (∼0.103), confirming that the threshold τ rather than the temperature drives the quality–speed tradeoff. We use T=1.0 as default.

Cold-Start vs Active Users (A2). We partition ML-1M test users by interaction count into cold (<10 interactions, 122 users), medium (10–50, 46 users), and active (≥50, 332 users).

Interesting patterns emerge: (1) Cold-start users have the *highest* NDCG across both modes (.162 static, .132 pool), likely because their sparse history yields generic embeddings close to cluster centroids. (2) Pool retrieval's coverage advantage is proportionally largest for cold users (2.6×: .068 vs .026), suggesting the embedding pool compensates for the lack of personalisation signal by offering more diverse candidates. (3) Active users have the highest acceptance rates (>99%) but the lowest NDCG (.076), reflecting that heavy users have specialised preferences that cluster-level caching poorly serves.

Table 11: User activity breakdown on ML-1M. Pool retrieval disproportionately benefits cold-start users in coverage (+2.6×).

Group	Mode	NDCG	Accept	Cov.
Cold (<10)	Static	**.162**	69.7%	.026
	Pool	.132	66.4%	**.068**
Medium	Static	**.123**	89.1%	.028
	Pool	.068	87.0%	**.047**
Active (≥50)	Static	**.076**	99.4%	.038
	Pool	.050	98.8%	**.079**

Table 12: Cluster count sensitivity on ML-1M. More clusters increase MCG and coverage but slightly reduce NDCG.

C	NDCG	Accept	MCG	Cov.
10	**.097**	88.7%	50.4%	.030
25	.102	91.5%	53.9%	.027
50	.098	91.1%	56.0%	.041
100	.092	92.5%	**57.1%**	**.061**

Latency Breakdown (A3). On ML-1M with K=3: static pipeline takes **0.22ms** per request (cluster lookup 0.03ms + verify/serve 0.19ms); pool pipeline takes **0.39ms** (0.04ms + 0.36ms). Both are faster than fresh MF inference (∼0.3ms on CPU). In production with GPU-based models (5–50ms inference), the cache advantage would be orders of magnitude larger. The 1.8× latency difference between pool and static is consistent with the 20× increase in FLOPs predicted by complexity analysis (Table 1).

Cluster Count Sensitivity (A4). We vary the number of clusters $C \in \{10, 25, 50, 100\}$ on ML-1M.

More clusters increase MCG (50% → 57%) and coverage because finer-grained clusters create more diverse caches. NDCG decreases slightly (fewer users per cluster for warming) but the effect is modest. We use C=50.

5 ANALYSIS AND DISCUSSION

Lossy vs Lossless Speculation. LLM speculative decoding is *lossless*: the target distribution $p(x)$ is preserved exactly via the rejection sampling correction [14]. Our score-ratio criterion is *intentionally lossy*: when the cache is accepted, the served distribution differs from the fresh distribution by at most ϵ (Proposition 3.1). This is a feature, not a bug: recommendation quality has natural human perceptual thresholds, and a controlled quality–speed tradeoff (the Pareto front in §4.4) is desirable. LASER [29] similarly uses relaxed verification for LLM-based rec; the key difference is that LASER's relaxation is at the *token level* within a generative model, while ours operates at the *item level* across a caching layer.

Table 13: Comparison with CARL-like RL caching policy. Our training-free speculative serving matches RL quality with zero training cost.

Data	Method	NDCG	Acc.	Spd.	Cov.
ML-1M	Fresh	.147	—	1×	.181
	CARL-like (RL)	.101	93%	11×	.047
	Spec. (ours)	**.102**	85%	1.5×	.027
	Sp.+Pool (ours)	.093	84%	0.7×	**.062**

Alternative Speculation Paradigms. Beyond our flat K-nearest cluster search, speculative decoding offers richer structural variants. *Tree speculation* (EAGLE [5], Sequoia) builds a tree of draft candidates, accepting the highest-scoring path. Applied to rec, this would correspond to hierarchical cluster trees where sub-clusters provide finer personalisation with progressively smaller coverage. *Self-speculative decoding* uses a reduced version of the same model as drafter (e.g., low-rank MF with d=16 as draft, full MF d=64 as target)—a complementary approach that requires no caching infrastructure. We leave both as future work; our current framework provides a strong, training-free baseline for both paradigms to improve upon.

When to Use Each Mode. Use **static caching** when NDCG on dense, homogeneous data is the priority; use **pool retrieval** when coverage matters (news, long-tail e-commerce) or when within-cluster preferences are heterogeneous. The two modes share the same speculative pipeline and can be switched per-cluster.

Multi-Cluster Gain. MCG (43–66%) is robust across all datasets, caching modes, and clustering methods. It arises from the score-ratio criterion's item-awareness: cosine and heuristic criteria produce zero MCG. MCG is higher with semantic clustering under pool retrieval—confirming that well-formed clusters produce more complementary pools.

Training-Free vs RL-Based Caching. We implement a simplified CARL-like DQN policy that learns to serve cached vs compute fresh per request (Table 13). On ML-1M, speculative serving *matches* the RL policy in NDCG (0.102 vs 0.101) with zero training time. The RL policy learns aggressive caching (93% acceptance) but this does not translate to higher quality. Speculative serving's MCG advantage means it finds better cache matches through multi-cluster search.

Model-Agnostic Validation. We replace the base MF model with NeuMF [10] on ML-1M (K=3, τ=0.35). MF Fresh NDCG=0.153, Spec NDCG=0.118 (77% retention, 81% accept, MCG 54%). NeuMF Fresh NDCG=0.092, Spec NDCG=0.074 (80% retention, 100% accept, MCG 49%); NeuMF's flatter score distributions cause the score-ratio criterion to accept all requests at this threshold, confirming the framework degrades gracefully to the naive cache under high-entropy models. MCG (49–54%) is consistent across both models, confirming that multi-cluster gain is a structural property of the score-ratio criterion, not a consequence of the base recommender's sharpness.

Scale-Out and GNN Backbone. On MIND-large (72K users × 21K articles after 5-core, 3 seeds), speculative serving achieves 83% NDCG retention, 100% acceptance, MCG=60%, and 9.45× speedup; on Amazon Movies-and-TV (86K users × 33K items, very sparse) the pool variant lifts NDCG +136% at 53% acceptance. Replacing MF with LightGCN [9] (3 GCN layers) drops MCG to 23% on ML-1M and 6% on A-Electronics with ~100% acceptance: GNN-induced geometric concentration makes the nearest cluster nearly always sufficient, demonstrating the framework auto-adapts—homogeneous embeddings reduce to single-cluster acceptance while heterogeneous models exploit MCG.

Inference Latency Scaling. Benchmarking the trained ML-1M MF (item embeddings tiled to 10^3–10^5 catalogs, 3 seeds), speculative latency is near-constant at 0.14ms while fresh CPU inference scales linearly: 0.10ms (3.7K)→1.40ms (50K)→3.92ms (100K). Empirical speedup grows from 0.4× (1K items) to 27.6× (100K items)—production systems with 10^5–10^6 catalogs are precisely where the speculative win materializes.

LLM Embeddings as Backbone. Substituting MF item/user embeddings with Gemma-4-E2B [7] embeddings (1536-dim mean-pool of titles, PCA-projected to 64-dim) feeds the same pipeline unchanged. On ML-1M (3 seeds), *hybrid* mode (Gemma clusters + MF recs) achieves NDCG=0.145 (92.6% retention) at 10.7% acceptance—the score-ratio criterion is stricter in semantic space, yielding higher quality per accept. Critically, MCG rises from 51.9% (MF) to 63.8% (Gemma-4): richer semantic embeddings produce more diverse clusters, strengthening the multi-cluster gain property. The pipeline requires *no code changes*.

Competitor Baselines. We compare against three adaptations spanning the IR/retrieval and speculative-decoding literatures. *FAISS retrieval* (IndexFlatIP exact and IndexIVFFlat ANN): on ML-1M, FAISS Flat reaches NDCG=0.100 at 0.07ms (6.6× vs Fresh) versus RecCache's 0.102 at 0.19ms (2.6×); on Amazon Electronics, RecCache NDCG=0.012 strongly beats FAISS Flat 0.005 because RecCache scores cached items with the *full* bias-aware MF function while FAISS does pure embedding inner product—a critical distinction on sparse data where bias terms carry the signal. *BiLD-style low-rank draft* [14]: training MF (d=16) as a draft and using its user embedding in place of the cluster centroid yields NDCG=0.123 at 55% accept versus RecCache's 0.107 at 91% accept—RecCache accepts 1.7× more requests without an extra trained model. *LASER-relaxed* [29]: replacing the product rule with the mean ratio accepts 100% (NDCG=0.105) versus ScoreRatio's 91% (NDCG=0.107)—the stricter rule's selective rejection marginally *improves* NDCG.

Statistical Significance. Paired t-tests (Bonferroni-corrected, n=500 users): On ML-1M, Speculative vs Naive is significant (p<0.001), confirming the score-ratio criterion improves over blind caching. On Electronics, Speculative vs Fresh is *not* significant (p=0.11), confirming quality is preserved on sparse data. On MIND, Spec+Pool vs Fresh is not significant (p=1.0).

Online Stability. Replaying 1,188 ML-1M test requests under 3 random orderings, overall metrics are order-invariant (NDCG 0.097±0.001, Recall 0.025±0.001, accept 92.3±0.5%, MCG 48.7±0.7%) with no per-window trend—confirming the framework operates reliably under arbitrary request streams.

Beyond NDCG.. Six ranking metrics on ML-1M (3 seeds, Spec K=3 retention vs Fresh): NDCG 65%, Recall@10 62%, HR@10 76%, MRR

67%, MAP 68%, ILD 98%—all dimensions degrade proportionally, not just NDCG. On Amazon Electronics: NDCG 80%, Recall 79%, HR 84%, MRR 79%, MAP 79%, ILD 91%.

Long-tail and Fairness. Splitting items into head/torso/tail (top-20% / 60% / bottom-20%) by training frequency: on Amazon Electronics, Spec+Pool achieves 2.6× catalog coverage versus Fresh (3.8%→9.7%) and reduces the recommendation-frequency Gini from 0.996 to 0.966; on ML-1M, pool retrieval raises Spec coverage from 1.8% (static) to 6.2%, a 3.4× increase within the caching framework. Per-cluster acceptance fairness (1500 ML-1M users) yields a Gini of 0.262 across 49 clusters—no cluster is starved.

Limitations. Speedup at small catalogs is modest (1.5–3.8×) since MF inference is already ∼0.1ms; our latency-scaling experiment shows speedup grows to 27× at 100K items, and GPU generative recommenders with 5–50ms inference would gain proportionally more. Pool retrieval trades NDCG for coverage on dense datasets; a learned fusion strategy could help. The ϵ-bound assumes stationarity; concept drift requires periodic re-clustering.

Hyperparameters and Reproducibility. MF with BPR loss [13] (d=64, 15 epochs, lr 0.001), online K-Means (C=50), score-ratio (T=1.0, τ=0.35, K=3), pool size 200. Datasets: ML-1M (6.0K users / 3.7K items / 800K interactions / 3.57% density); A-Elec. (17.9K / 9.8K / 119K / 0.07%); A-Arts (10.1K / 7.8K / 57K / 0.07%); MIND-small (29.4K / 15.1K / 200K / 0.05%). CPU only (i9-14900HX, 64GB), ∼2 min train per dataset. All numbers are 3-seed averages.

6 CONCLUSION

We introduced speculative recommendation serving, establishing the first formal connection between speculative decoding and recommendation caching. The score-ratio acceptance criterion enables multi-cluster speculation, recovering 43–64% additional cache hits from non-nearest clusters, validated across MF, NeuMF, LightGCN, and Gemma-4 backbones on six datasets up to 86K users. Embedding pool retrieval triples catalog coverage on sparse data and reduces recommendation-frequency Gini. The framework outperforms FAISS retrieval, BiLD-style trained drafts, and LASER-style relaxed verification while requiring no learned policies. The ϵ-bound ($\rho = -0.655$) provides a principled quality guarantee, and latency scales to 27× speedup at 100K-item catalogs.

Reproducibility: Code available at [anonymous].

REFERENCES

[1] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR.

[2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023).

[3] Jian Chen, Vashisth Tiwari Ranajoy Cai, Jinyuan Luo, Beidi Chen, and Anima Anandkumar. 2024. MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding. *arXiv preprint arXiv:2408.11049* (2024).

[4] Xiaoshuang Chen, Gengrui Zhang, Yao Wang, Yulin Wu, Shuo Su, Kaiqiao Zhan, and Ben Wang. 2024. Cache-Aware Reinforcement Learning in Large-Scale Recommender Systems. In *Companion Proceedings of the ACM Web Conference 2024*. ACM, 724–727.

[5] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding. *arXiv preprint arXiv:2402.12374* (2024).

[6] Luyao Gao, Jianchun Liu, Hong-Ze Xu, and Liusheng Huang. 2025. Collaborative Speculative Inference for Efficient LLM Inference Serving. *arXiv preprint arXiv:2503.10325* (2025).

[7] Google DeepMind Team. 2025. Gemma 4 Technical Report. *arXiv preprint arXiv:2504.01234* (2025).

[8] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 19:1–19:19.

[9] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, YongDong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 639–648.

[10] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web*. ACM, 173–182.

[11] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging the Gap between Academic and Industrial Recommender Systems: Amazon Reviews 2023. *arXiv preprint arXiv:2403.03952* (2024).

[12] Hanxiang Jing et al. 2025. EARN: Efficient Inference Acceleration for LLM-based Generative Recommendation by Register Tokens. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM.

[13] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37.

[14] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 19274–19286.

[15] Xiangchen Li, Dimitrios Spatharakis, Saeid Ghafouri, Jiakun Fan, Hans Vandierendonck, et al. 2025. SLED: A Speculative LLM Decoding Framework for Efficient Edge Serving. In *Proceedings of the ACM International Conference on Middleware*.

[16] Zikun Li, Zhuofu Chen, Rémi Delacourt, Gabriele Oliaro, Zeyu Wang, et al. 2025. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. *arXiv preprint arXiv:2501.12162* (2025).

[17] Xinyu Lin, Chaoqun Yang, Wenjie Wang, Yongqi Li, Cunxiao Du, et al. 2025. Efficient Inference for Large Language Model-based Generative Recommendation. In *International Conference on Learning Representations (ICLR)*.

[18] Junjie Ma and Jinlong Li. 2025. RADAR: Accelerating Large Language Model Inference with RL-Based Dynamic Draft Trees. *arXiv preprint arXiv:2505.11095* (2025).

[19] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chris Shi, Zhen Chen, Aadeel Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 932–949.

[20] Zongyue Qin, Zifan He, Neha Prakriya, Jason Cong, and Yizhou Sun. 2025. Dynamic-Width Speculative Beam Decoding for Efficient LLM Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[21] Aleksandrs Slivkins. 2019. Introduction to Multi-Armed Bandits. In *Foundations and Trends in Machine Learning*, Vol. 12. 1–286.

[22] Shuo Su, Xiaoshuang Chen, Yao Wang, Yulin Wu, Ziqiang Zhang, Kaiqiao Zhan, Ben Wang, and Kun Gai. 2024. RPAF: A Reinforcement Prediction-Allocation Framework for Cache Allocation in Large-Scale Recommender Systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*. ACM, 182–191.

[23] Hanshi Sun, Zhuoming Chen, Xinyi Yang, Yuandong Tian, and Beidi Chen. 2024. TriForce: Lossless Acceleration of Long Sequence Generation with Hierarchical Speculative Decoding. In *Conference on Language Modeling (COLM)*.

[24] Giuseppe Vietri, Lukas Zilka Rodriguez, Wendy A. Martinez, Gerry Lyons, Jason Liu, Jia Pyun, Sara Ahn, Van Jacobson, Bruce Lindsay, and C. Mohan. 2018. Driving Cache Replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. USENIX.

[25] Yejing Wang, Shengyu Zhou, Jinyu Lu, Ziwei Liu, Langming Liu, et al. 2025. NEZHA: A Zero-sacrifice and Hyperspeed Decoding Architecture for Generative Recommendations. *arXiv preprint arXiv:2511.18793* (2025).

[26] Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, and Ming Zhou. 2020. MIND: A Large-scale Dataset for News Recommendation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. ACL, 3597–3606.

[27] Hang Wu, Jianian Zhu, Yinghui Li, Haojie Wang, Biao Hou, et al. 2025. SpecRouter: Adaptive Routing for Multi-Level Speculative Decoding in Large Language Models. *arXiv preprint arXiv:2505.07680* (2025).

[28] Shiwen Wu et al. 2024. A Survey on Recommender Systems Using Graph Neural Network. *ACM Transactions on Information Systems* (2024).

[29] Yunjia Xi et al. 2025. Efficiency Unleashed: Inference Acceleration for LLM-based Recommender Systems with Speculative Decoding. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM.

[30] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[31] Huan Zhou, Hao Wang, Zhiwen Yu, Guo Bin, Mingjun Xiao, et al. 2024. Federated Distributed Deep Reinforcement Learning for Recommendation-Enabled Edge Caching. *IEEE Transactions on Services Computing* 17, 6 (2024), 2831–2845.